INFORME DE LABORATORIO #2

Expresiones Regulares con JFLAP

Antecedente	Información
Asignatura	Teoría de la Computación INFO1148
Carrera	Ingeniería Civil en Informática
Docente	M. Lévano
Integrantes	Martín Gonzalez, Oscar Barra
Fecha	11 / 10 / 2026

1. Introducción

El propósito de este laboratorio es validar de forma práctica la equivalencia entre **Expresiones Regulares**, **Autómatas Finitos No deterministas** y **Autómatas Finitos Deterministas**. Mediante **JFLAP** se interpretan y diseñan expresiones regulares sobre alfabetos y restricciones, y se convierten a AFN por construcción de Thompson y a AFD por construcción de subconjuntos, y se comprueba su comportamiento con cadenas aceptadas, rechazadas y casos límite.

2. Desarrollo

2.1 Pregunta 1 – Interpretación y validación de una expresión regular

a) Lenguaje representado:

Dado el alfabeto: $\sum$ **= {0,1}**, y la ER: **(0+1)*01**, el alfabeto que representa a la expresión regular es: **L = { w ∈ {0,1}* | w acabe con la subcadena 01 }.** El cual acepta como cadenas válidas, sólo aquellas que utilicen el alfabeto establecido y acaben con la subcadena 01.

b) Resultados obtenidos

Cadena	Predicción	Resultado JFLAP	¿Coinciden?
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

c) Evidencia en JFLAP

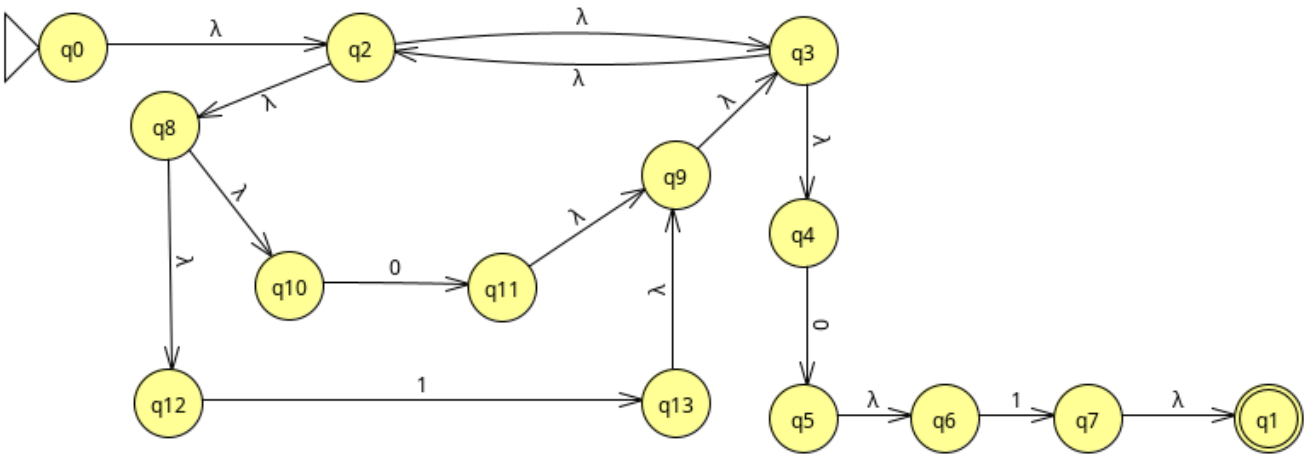


Figura 1. Autómata finito no determinista para la ER: **(0+1)*01**

d) Análisis

1101 es aceptada porque cumple con la restricción impuesta en el lenguaje y el sufijo 01 llega al estado final. Mientras que 0100 es rechazada porque termina con la subcadena 00, y aunque contiene el prefijo 01, la ER exige la subcadena 01 al final, por lo que no existe un camino al estado terminal con el último 0.

2.2 Pregunta 2 – Diseño de una expresión regular

a) Expresión regular propuesta

Lenguaje: **L = { w ∈ {a,b}* | w comienza con a y termina con b }**

Expresión regular: **a(a+b)*b**

b) Explicación

La expresión regular: a(a+b)*b, cumple con las restricciones definidas en el lenguaje, porque la ER logra generalizar todas las posibles cadenas válidas. Dado que:

- La **a** del inicio, obliga a todas las cadenas comenzar con ese carácter.
- El operador central **(a+b)***, permite cualquier combinación intermedia de caracteres que pertenecen al alfabeto.
- La **b** del final, obliga a todas las cadenas terminal con ese carácter.

c) Casos de prueba

Cadena	Resultado esperado	Resultado JFLAP
ab	Aceptada	Aceptado
aab	Aceptada	Aceptado
abb	Aceptada	Aceptado

abab	Aceptada	Aceptado
apkb	Rechazada	Rechazado
ba	Rechazada	Rechazado
aaba	Rechazada	Rechazado
b	Rechazada	Rechazado
a	Rechazada	Rechazado
λ	Rechazada	Rechazado

d) Evidencia en JFLAP

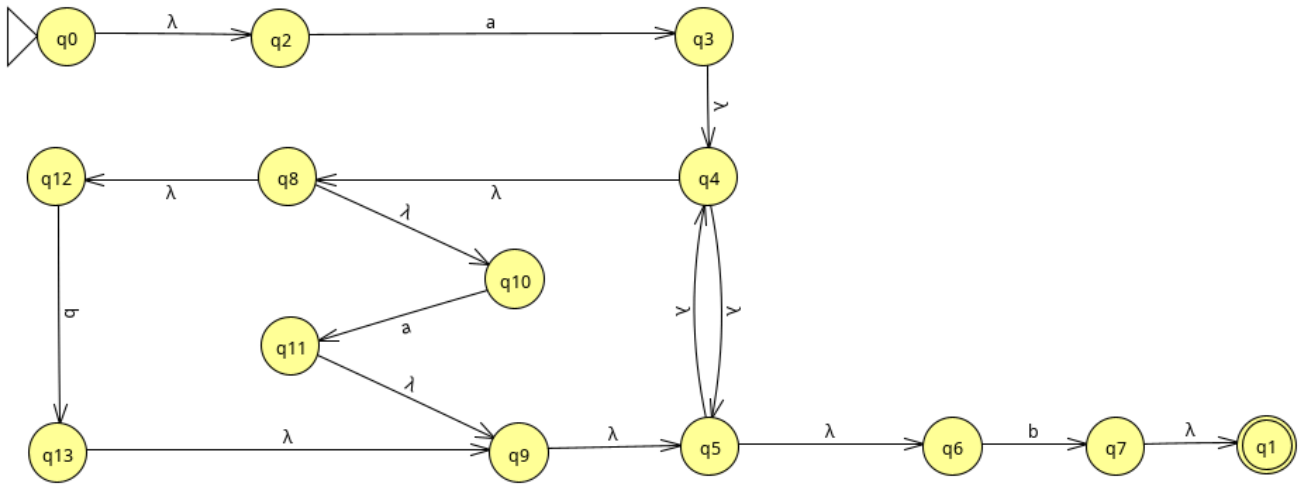


Figura 2. Autómata finito no determinista para la expresión regular: **a(a+b)*b**

e) Análisis

Dadas las cadenas de pruebas y los resultados esperados, al crear el AFN a partir de la ER propuesta, se pudieron comprobar los resultados usando la herramienta JFLAP, logrando comprobar que el 100% de los resultados coinciden con lo esperado, demostrando que la expresión regular: a(a+b)*b, describe correctamente el lenguaje y la restricción establecida.

2.3 Pregunta 3 – Conversión ER → AFN → AFD

a) Expresión regular propuesta

Lenguaje: **L = { w ∈ {0,1}* | w contiene la subcadena 101 }**

ER = **(0+1)*101(0+1)***

b) Características de los autómatas

Característica	AFN	AFD
Número de estados	24	9
Número de transiciones	29	18
Transiciones λ	22	0
Determinista	No	Si

c) Pruebas realizadas

Cadena	Resultado AFN	Resultado AFD	¿Coinciden?
101	Aceptado	Aceptado	Si
001011	Aceptado	Aceptado	Si
1109101	Rechazado	Rechazado	Si
111001	Rechazado	Rechazado	Si
000101	Aceptado	Aceptado	Si
10101	Aceptado	Aceptado	Si
10	Rechazado	Rechazado	Si
λ	Rechazado	Rechazado	Si

d) Evidencias

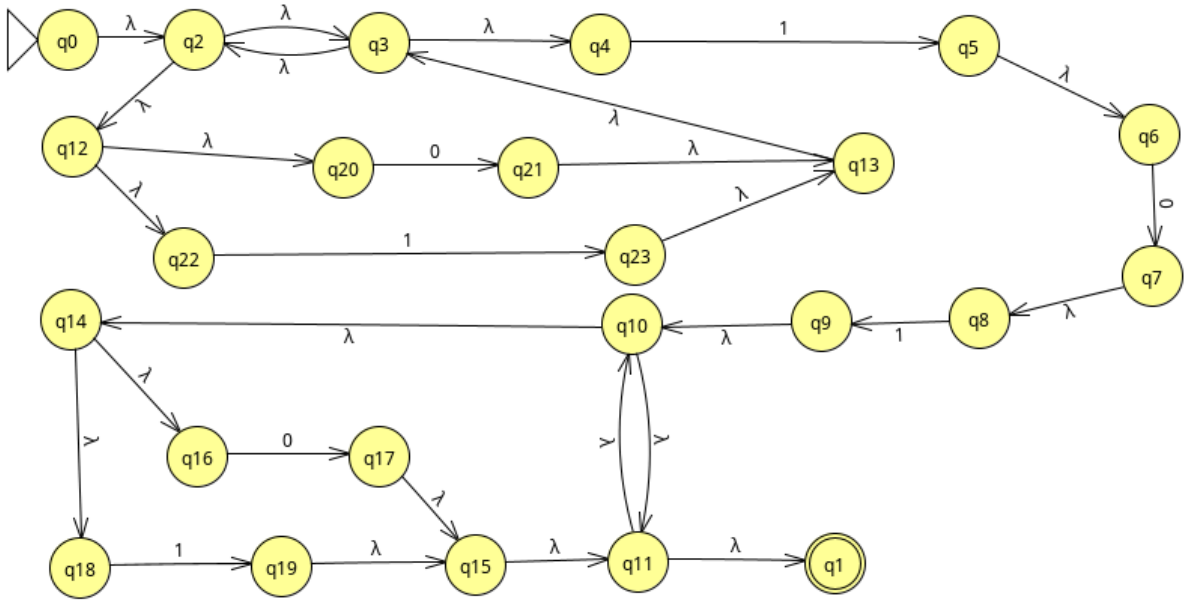


Figura 3. AFN generado a partir de la ER: **(0+1)*101(0+1)***

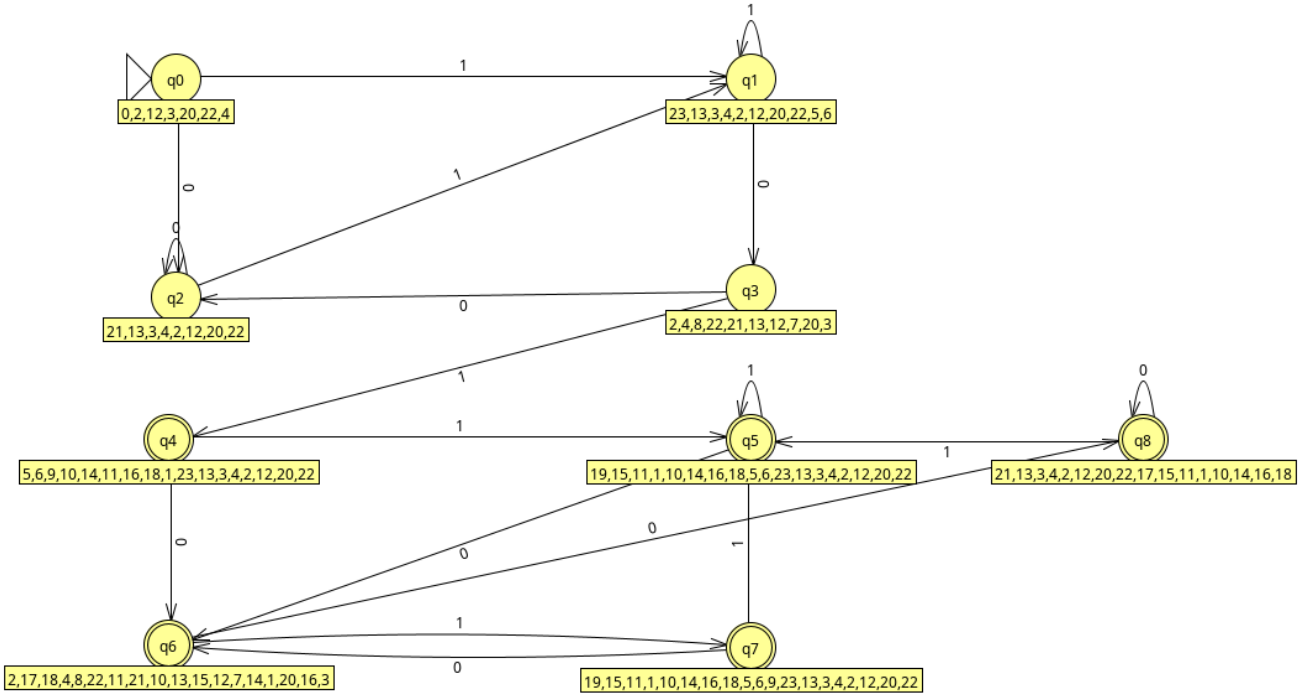


Figura 4. AFD generado a partir del AFN anterior

e) Análisis

Dadas las cadenas de pruebas anteriores, se observó que tanto el AFN y el AFD coinciden en el resultado de la validación sobre una misma cadena.

En ambos autómatas las entradas consideradas válidas, son aquellas que cumplen la restricción establecida en el lenguaje: { w ∈ {0,1}* | w contiene la subcadena 101 }, y que utilizan caracteres pertenecientes al alfabeto.

Y las cadenas consideradas por ambos autómatas como inválidas, se debe a que no cumplen la restricción de tener la subcadena 101, o por que contienen caracteres que no pertenecen al alfabeto definido.

Y por el comportamiento anterior se refleja el principio de equivalencia, que establece que todo AFN puede ser convertido en un AFD mediante el algoritmo de construcción de subconjuntos, de tal manera que ambos autómatas reconozcan el mismo conjunto de cadenas.

2.4 Pregunta 4 – Diseño y validación

a) Expresión regular propuesta

Alfabeto: $\sum$ **= {a,b}**

Lenguaje: **L = { w ∈ {a,b}* | w contiene al menos una a y termina en bb }**

ER = **(a+b)*a(a+b)*bb**

b) Justificación

Dado el alfabeto y las restricciones que debe cumplir la expresión regular, se propone la siguiente ER: (a+b)*a(a+b)*bb, la cual al descomponerla por partes, se puede explicar de la siguiente manera:

- El primer **(a+b)***, permite aceptar cadenas que empiezan por los caracteres **a** y **b**, o por el carácter vacío **λ**.
- El carácter **a**, asegura que existe al menos un símbolo 'a' dentro de la cadena.
- El segundo **(a+b)***, permite caracteres intermedios que pertenezcan al alfabeto.
- Por último, **bb** obliga a cada cadena a finalizar con la subcadena 'bb' para que sea considerada válida.

Por lo anterior, la ER propuesta es capaz de identificar y validar cadenas que cumplan el patrón anterior.

c) Casos de prueba

Cadena	Resultado esperado	AFN	AFD
abb	Aceptada	Aceptada	Aceptada
aabb	Aceptada	Aceptada	Aceptada
babbb	Aceptada	Aceptada	Aceptada
ababbb	Aceptada	Aceptada	Aceptada
abbb	Aceptada	Aceptada	Aceptada
bb	Rechazada	Rechazada	Rechazada
aaa	Rechazada	Rechazada	Rechazada
aba	Rechazada	Rechazada	Rechazada
baba	Rechazada	Rechazada	Rechazada
bbbb	Rechazada	Rechazada	Rechazada
ab	Rechazada	Rechazada	Rechazada
λ	Rechazada	Rechazada	Rechazada

d) Evidencias

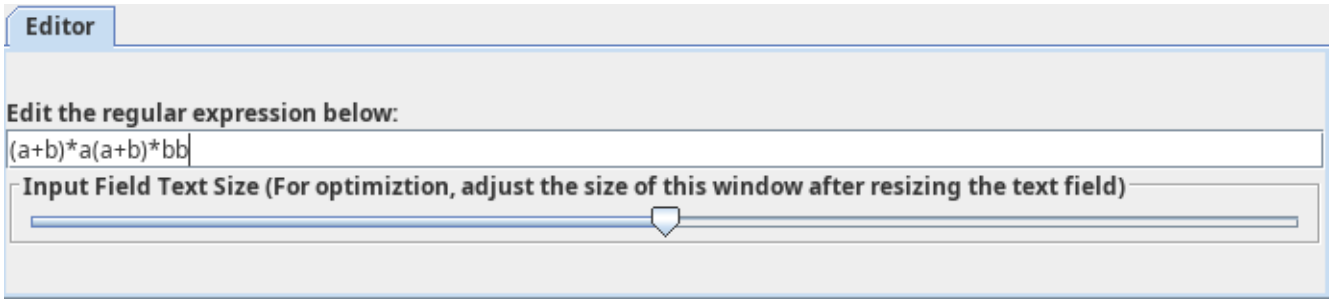


Figura 5. Expresión regular en JFLAP

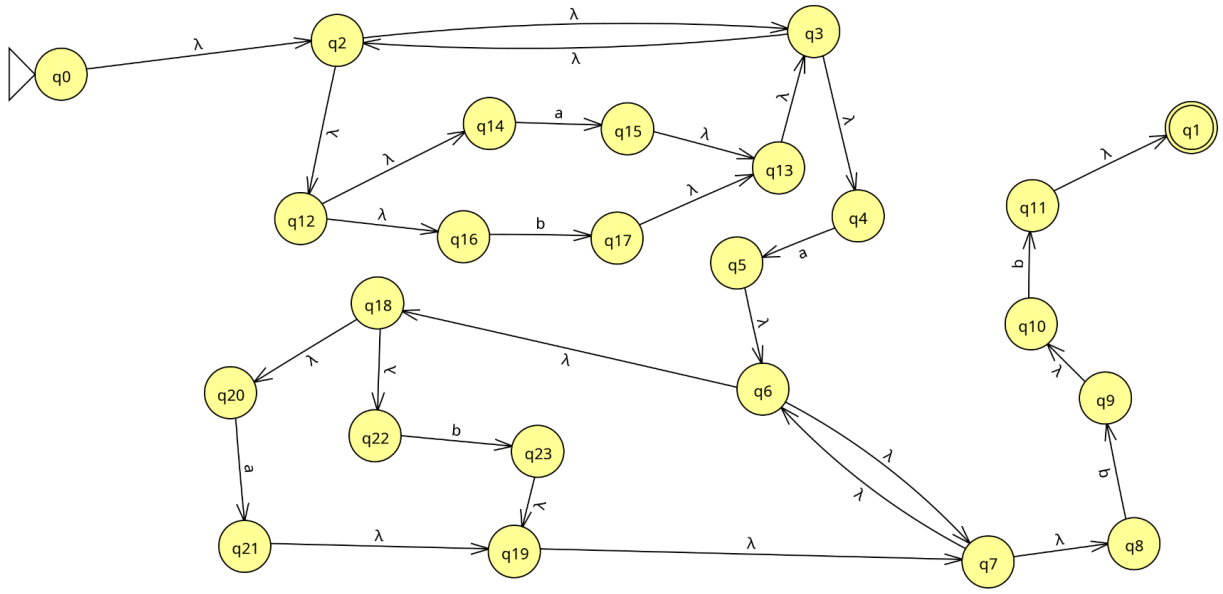


Figura 6. Autómata finito no determinista para la ER: **(a+b)*a(a+b)*bb**

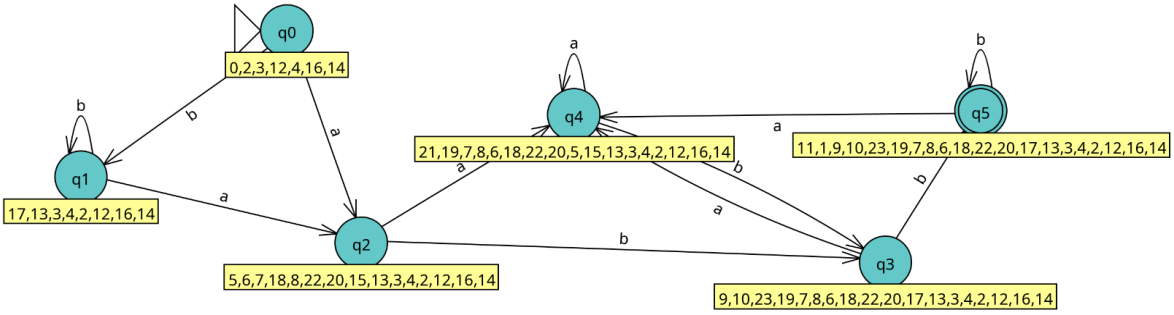


Figura 7. AFD generado a partir del AFN anterior

e) Reflexión: ¿Expresión regular, AFN y AFD poseen distinta capacidad de reconocimiento o son formas diferentes de representar una misma clase de lenguajes?

Los tres modelos describen el mismo lenguaje, funcionando como perspectivas complementarias.

- **Expresión regular:** Es una representación declarativa y compacta, ideal para describir patrones de forma abstracta.
- **AFN:** Describe un modelo que puede explorar múltiples caminos a la vez para ver si una cadena cumple con un patrón.
- **AFD:** Describe un modelo estrictamente determinista, que tiene un único camino posible.

Las transformaciones entre ellos: ER ⟶ AFN ⟶ AFD, logra conservar el lenguaje original, porque existen métodos matemáticos para realizar las transformaciones:

- De ER a AFN, mediante el **algoritmo de Thompson.**
- De AFN a AFD, mediante la **construcción de subconjuntos.**
- De AFD o AFD a ER, mediante la **eliminación de estados.**

3. Conclusiones

Durante el desarrollo del laboratorio, se pudo demostrar de forma práctica que una expresión regular, un AFN y un AFD son formas equivalentes de representar un mismo lenguaje.

El flujo de trabajo nos invitó a comprender, investigar y realizar predicciones sobre los resultados esperados sobre distintas cadenas de pruebas, con el fin de evaluar la robustez de las ER propuestas para identificar patrones sobre un alfabeto y sus restricciones.

Por lo que este laboratorio nos permitió diseñar y probar distintas expresiones regulares en la herramienta de JFLAP, además de ayudarnos a recordar y reforzar los conocimientos que teníamos sobre los autómatas finitos no deterministas y autómatas finitos deterministas.

4. Archivos adjuntos

Pregunta 1	Pregunta 2	Pregunta 3	Pregunta 4
P1_AFN.jff P1_ER.jff	P2_AFN.jff P2_ER.jff	P3_AFN.jff P3_AFD.jff P3_ER.jff	P4_AFN.jff P4_AFD.jff P4_ER.jff

Enlace de Github: https://github.com/Zgonzalez23/Lab2.git

5. Anexo

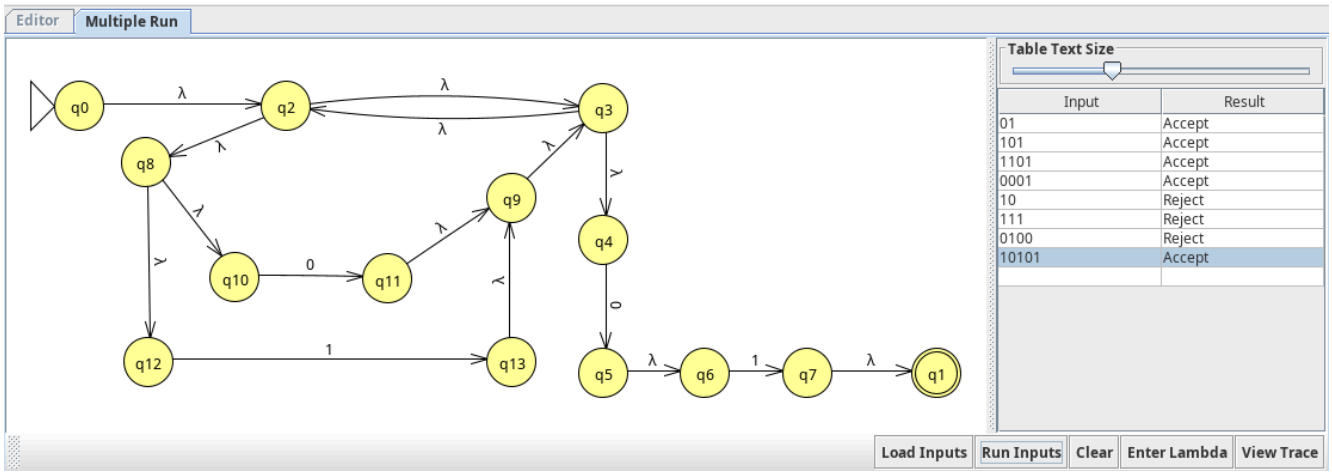


Figura 8. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 1

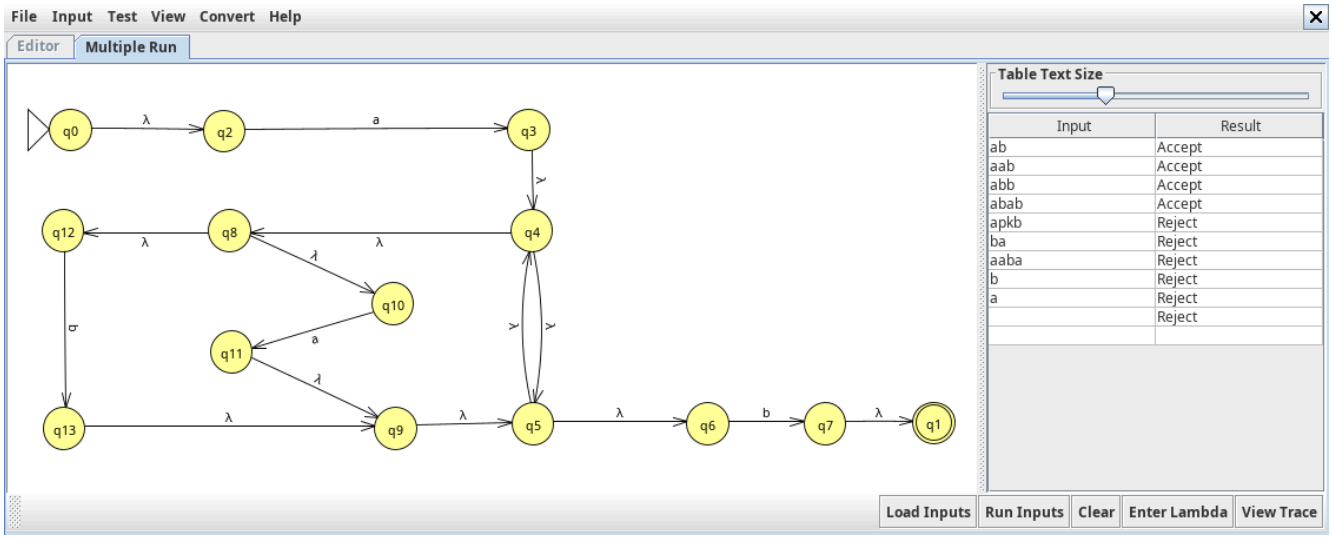


Figura 9. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 2

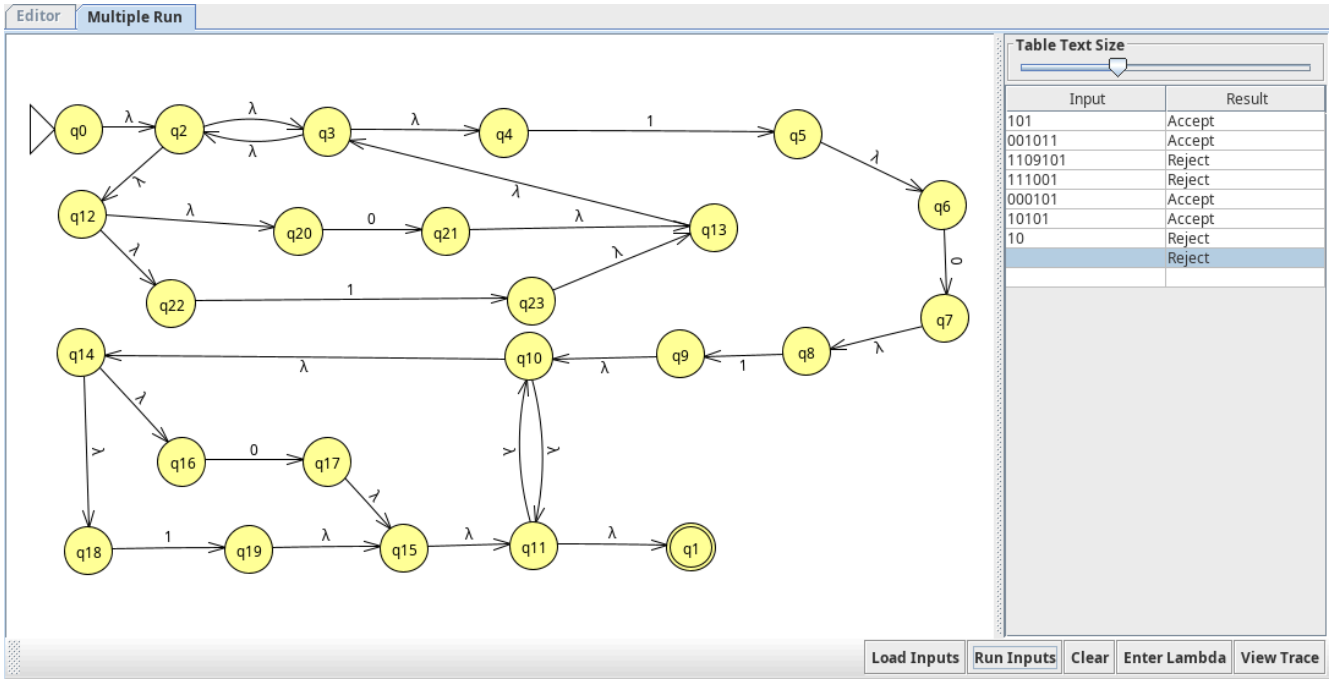


Figura 10. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 3 en el AFN

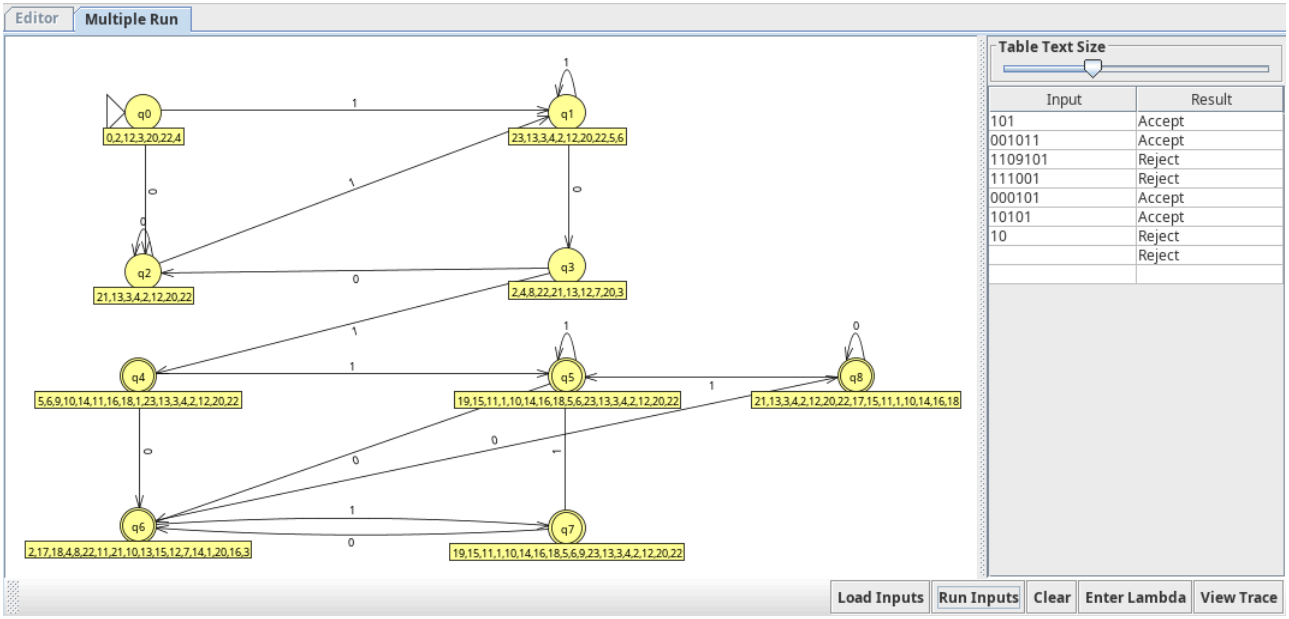


Figura 11. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 3 en el AFD

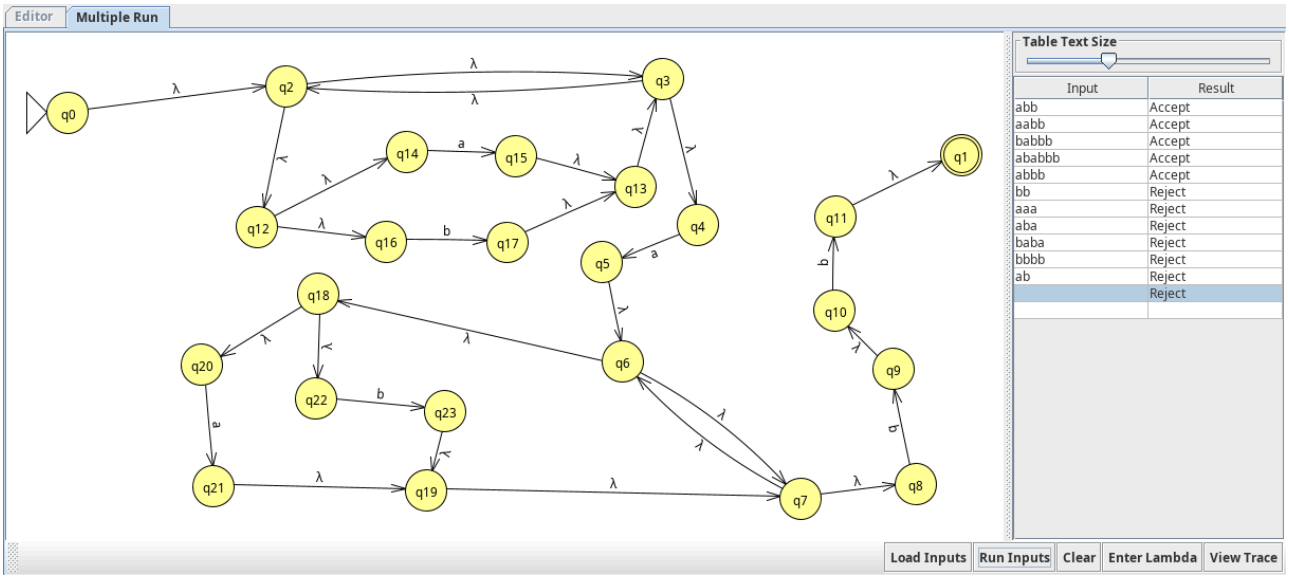


Figura 12. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 4 en el AFN

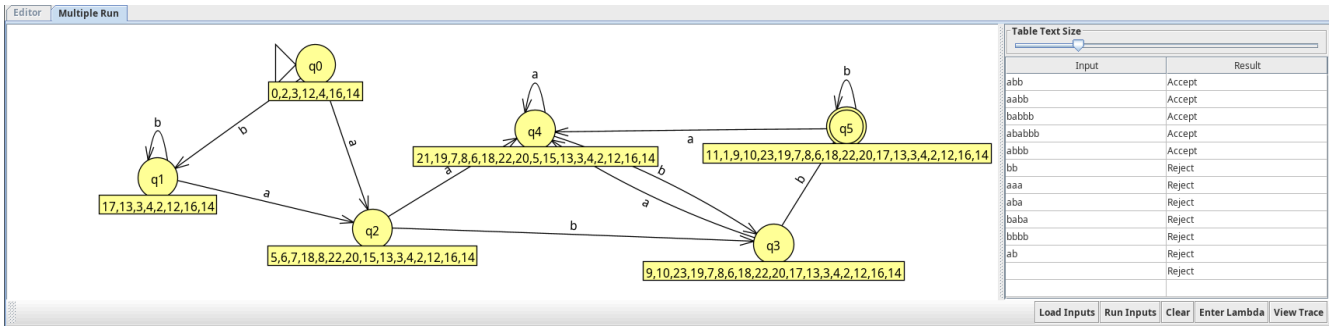


Figura 13. Ejecución y resultados de la validación de las cadenas de prueba para la pregunta 4 en el AFD